PROJECT DEVELOPMENT PLAN: Rider Community & Safety Platform

Document Type: Technical & Execution Strategy

Timeline: 22 Days (20 Days Development + 2 Days Testing)

Team Size: 3 Developers (Frontend, Backend, API/QA)

1. Project Overview

This platform is a dual-purpose rider ecosystem combining a **social network** (feeds, reels, profiles) with a **real-time safety & ride coordination tool** (live maps, WebSockets, geofenced alerts, and secure chat).

Tech Stack Definition

- **Frontend:** React Native (Expo Go) - using Expo Router
- **Backend:** Node.js (Express/Hapi)
- **Database:** PostgreSQL (Text/Relational Data)
- **Storage:** Cloudflare R2 (Media/Object Storage - S3 compatible)
- **Hosting/Infrastructure:** VPS (Virtual Private Server)
- **Testing/API:** Postman
- **3rd Party SDKs:** Firebase (Auth), Google Maps (Routing/Places)
- **Security & Chat:** End-to-End Encryption (E2EE) for chat messages. (Chat payloads are encrypted on the client device and stored securely in the PostgreSQL database under the user's records, ensuring the server cannot read message contents).

2. UI/UX Strategy & Color Palette

Color Palette

The app uses a "Dark Mode First" approach, which reduces glare while riding and saves phone battery. The provided palette has been adapted to create a warm, premium dark mode aesthetic.

- **Primary Accent:** Vibrant Red (#D84040) - Used for primary CTAs, active button states, and map markers.
- **Secondary Accent:** Deep Red (#A31D1D) - Used for pressed states, secondary buttons, and deep highlights.
- **Background Base:** Warm Deep Charcoal (#181515) - Main app background, heavily darkened to support dark mode while maintaining a warm undertone to complement the reds.

- **Surface / Cards:** Warm Dark Grey (#2B2525) - Used for feed cards, bottom sheets, dialogs, and input backgrounds to create depth against the base background.
- **Text (Primary):** Warm Cream (#F8F2DE) - Used for primary headings, titles, and main body text. Provides high contrast without the harshness of pure white.
- **Text (Secondary):** Sand Beige (#ECDCBF) - Used for secondary text, subtitles, muted icons, and subtle borders.
- **Success/Safe:** Muted Green (#4CAF50) - Used for "In Zone" status or successful actions.
- **Danger/Alert:** Vibrant Red (#D84040) - The primary accent natively doubles as the "Out of Zone" or SOS alert color.

UI Design Format & UX Explanation

- **High Contrast & Large Tap Targets:** Riders often have phones mounted on their handlebars or wear gloves. Buttons must be minimum 48x48dp.
- **Navigation:** Bottom Tab Navigation for core areas (Home, Explore, Ride, Profile) + Floating Action Button (FAB) for quick content creation/ride start.
- **Live Map Focus:** The map UI will be immersive (full-screen map with translucent floating overlay cards for rider stats) to maximize situational awareness.
- **Bottom Sheets:** Use bottom sheets instead of new pages for quick actions (like adding a bike or commenting) to maintain context.

3. Screen Breakdown & Content Mapping

Total Estimated Screens: 14

| **#** | **Screen Name** | **Content & Features** |

| 1 | **Splash / Auth Check** | App logo, loading spinner, silent token validation. |

| 2 | **Login & Signup** | Email/Password inputs, Firebase Auth integration, T&C checkbox. |

| 3 | **Profile & Garage Setup** | Avatar upload, Bio, "Add Bike" form (Brand, Model, Year, Photo). |

| 4 | **Home Feed** | Scrollable list of Posts, horizontal scrolling Reels/Stories, Like/Comment/Share buttons. |

| 5 | **Explore** | Search bar, user list, suggested rooms, trending music/clips. |

| 6 | **Content Creator** | Camera view, gallery picker, caption input, tag rider/bike, publish toggle. |

| 7 | **Communities / Rooms** | List of active rooms, "Create Room" button, "Join with Code" input. |

| 8 | **Room Details** | Room title, Captain badge, member list, "Start Live Ride" button, Chat

preview. |

| 9 | **Live Map (Ride Engine)** | Full-screen map, rider avatars on map, speed/distance overlay, "Leave Ride" button, SOS/Alert toast notifications. |

| 10 | **Room Chat (E2EE)** | Secure chat UI, text input, media attachment, system messages. (Messages encrypted/decrypted locally). |

| 11 | **Profile Dashboard** | User stats (Miles, Avg Speed), Badges/Achievements grid, Garage/Bike list. |

| 12 | **Followers / Tracking** | Lists of Trackers (Followers) and Tracking (Following). |

| 13 | **Notifications** | Alerts list (system alerts, out-of-zone warnings, likes, comments). |

| 14 | **Settings** | Privacy toggles (location visibility), account deletion, change password, logout. |

4. Code Reusability & Architecture Plan

Frontend Reusability (React Native / Expo)

- **Global Contexts:** AuthContext, RideContext, E2EEncryptionContext (manages local key pairs).
- **Service Wrappers:** Dedicated files for API calls to prevent logic from bleeding into UI components (e.g., FeedService.js, GarageService.js).
- **Custom Hooks:** * useLocation(): Reusable hook to track GPS coordinates.
    - useWebSocket(roomId): Reusable hook for live ride room connections.
    - useCloudflareUpload(): Reusable hook for uploading media to R2.
- **UI Components:** <PrimaryButton>, <BikeCard>, <Avatar>, <FeedPost>, <FormInput>.

Backend Reusability (Node.js)

- **Layered Architecture:** Routes -> Controllers -> Services -> DAOs.
- **Middlewares:** * requireAuth: Validates JWT via Firebase Admin.
    - validateBody(schema): Reusable payload validation using Joi or Zod.
- **Shared Functions:** * generateRideReport(rideId): Used by both manual ride logging and live-ride completion.
    - geoFenceCheck(lat, lon, polygon): Math utility for location bounds.

5. Complete Database Schema (PostgreSQL)

The application relies on a strictly relational schema to connect identity, content, live rides, and secure communications. We use UUID for all primary keys to ensure global uniqueness and prevent enumeration attacks.

5.1 Identity & Profile Data

rider_account

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Unique user identifier
firebase_uid	VARCHAR(128)	UNIQUE, NOT NULL	Linked Firebase auth ID
email	VARCHAR(255)	UNIQUE, NOT NULL	User's email address
username	VARCHAR(50)	UNIQUE, NOT NULL	Public @handle
name	VARCHAR(100)	NOT NULL	Display name
bio	TEXT	NULL	Short user biography
profile_image_url	VARCHAR(255)	NULL	Cloudflare R2 URL
banner_image_url	VARCHAR(255)	NULL	Cloudflare R2 URL
total_miles	DECIMAL(10,2)	Default: 0.00	Accumulated ride mileage
created_at	TIMESTAMPTZ	Default: NOW()	Account creation time
updated_at	TIMESTAMPTZ	Default: NOW()	Last profile update

user_bike (Garage)

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Unique bike identifier
rider_id	UUID	FK -> rider_account.id	Owner of the bike

brand	VARCHAR(100)	NOT NULL	e.g., Honda, Ducati
model	VARCHAR(100)	NOT NULL	e.g., CBR600RR
year	INTEGER	NOT NULL	Manufacture year
bike_image_url	VARCHAR(255)	NULL	Photo of the bike
is_primary	BOOLEAN	Default: FALSE	Default bike used for rides

5.2 End-to-End Encrypted Communications

user_encrypted_chats

Data is stored encrypted. The backend has zero knowledge of the message content.

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Message ID
sender_id	UUID	FK -> rider_account.id	Rider who sent the message
receiver_id	UUID	FK -> rider_account.id, NULL	Target user (for direct messages)
room_id	UUID	FK -> community.id, NULL	Target room (for group chats)
encrypted_payload	TEXT	NOT NULL	AES-GCM encrypted message string
iv	VARCHAR(255)	NOT NULL	Initialization Vector used for decryption
created_at	TIMESTAMPTZ	Default: NOW()	Time message was sent

5.3 Social Layer Data

feed_post

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Post ID
rider_id	UUID	FK -> rider_account.id	Author
caption	TEXT	NULL	Accompanying text
media_url	VARCHAR(255)	NULL	Image or video R2 URL
media_type	VARCHAR(20)	NULL	IMAGE, VIDEO, or TEXT
created_at	TIMESTAMPTZ	Default: NOW()	Time posted

feed_comment & feed_like

Table	Column	Type	Constraints
feed_comment	id	UUID	PK
	post_id	UUID	FK -> feed_post.id, ON DELETE CASCADE
	rider_id	UUID	FK -> rider_account.id
	comment_text	TEXT	NOT NULL
	created_at	TIMESTAMPTZ	Default: NOW()
feed_like	id	UUID	PK
	post_id	UUID	FK -> feed_post.id,

			ON DELETE CASCADE
	rider_id	UUID	FK -> rider_account.id
	created_at	TIMESTAMPTZ	Default: NOW()

clip (Reels)

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Clip ID
rider_id	UUID	FK -> rider_account.id	Creator
video_url	VARCHAR(255)	NOT NULL	Video R2 URL
song_id	VARCHAR(100)	NULL	External music/audio reference
created_at	TIMESTAMPTZ	Default: NOW()	Time uploaded

tracker (Follower System)

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Tracker mapping ID
follower_id	UUID	FK -> rider_account.id	Person doing the following
following_id	UUID	FK -> rider_account.id	Person being followed
created_at	TIMESTAMPTZ	Default: NOW()	Follow timestamp

5.4 Ride Coordination & Safety Data

community (Rooms)

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Room ID
creator_id	UUID	FK -> rider_account.id	Initial Captain / Owner
name	VARCHAR(100)	NOT NULL	Room display name
password_hash	VARCHAR(255)	NULL	Optional PIN for private rooms
is_active	BOOLEAN	Default: TRUE	Active or archived state
created_at	TIMESTAMPTZ	Default: NOW()	Creation time

community_member

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Membership ID
community_id	UUID	FK -> community.id	Room reference
rider_id	UUID	FK -> rider_account.id	User reference
role	VARCHAR(20)	Default: MEMBER	CAPTAIN or MEMBER
joined_at	TIMESTAMPTZ	Default: NOW()	Time joined

ride & ride_participant (Live Ride Tracking)

Table	Column	Type	Constraints	Description
ride	id	UUID	PK	Ride session ID
	community_id	UUID	FK -> community.id	Room hosting the ride
	status	VARCHAR(20)	Default: PLANNING	PLANNING, ACTIVE, COMPLETED
	start_time	TIMESTAMPTZ	NULL	Actual start time
	end_time	TIMESTAMPTZ	NULL	Completion time
	route_polygon	JSONB	NULL	GeoJSON for zone tracking
ride_participant	id	UUID	PK	Session member ID
	ride_id	UUID	FK -> ride.id	Ride reference
	rider_id	UUID	FK -> rider_account.id	Rider reference
	status	VARCHAR(30)	Default: IN_ZONE	IN_ZONE, OUT_OF_ZONE, SOS
	distance	DECIMAL	Default: 0.00	Real-time session distance

ride_report (Post-Ride Telemetry)

Column	Type	Constraints	Description
id	UUID	PK, Default:	Report ID

		uuid_generate_v4()	
ride_participant_id	UUID	FK -> ride_participant.id	Source participant session
top_speed	DECIMAL(5,2)	Default: 0.00	Max speed logged (mph/kmh)
average_speed	DECIMAL(5,2)	Default: 0.00	Avg speed calculated
distance	DECIMAL(10,2)	Default: 0.00	Final distance recorded
badges_earned	JSONB	Default: []	JSON array of badge IDs awarded

notification

Column	Type	Constraints	Description
id	UUID	PK, Default: uuid_generate_v4()	Notification ID
rider_id	UUID	FK -> rider_account.id	Recipient
title	VARCHAR(100)	NOT NULL	Push title
body	TEXT	NOT NULL	Push body / context
type	VARCHAR(30)	NOT NULL	ALERT, SOCIAL, SYSTEM
is_read	BOOLEAN	Default: FALSE	Read status
created_at	TIMESTAMPTZ	Default: NOW()	Timestamp

6. Folder Structure

Frontend Structure (/frontend - React Native / Expo)

/frontend

```
├── app/                    # Expo Router Pages
│   ├── (tabs)/             # Bottom Tab Navigator Layout
│   │   ├── _layout.tsx
│   │   ├── index.tsx       # Home Feed
│   │   ├── explore.tsx
│   │   ├── ride.tsx        # Live Map / Enroute
│   │   └── profile.tsx     # User Dashboard
│   ├── auth/               # Authentication Flow
│   │   ├── _layout.tsx
│   │   ├── login.tsx
│   │   └── signup.tsx
│   ├── room/
│   │   └── [id].tsx        # Dynamic Room Details & E2E Chat
│   ├── _layout.tsx         # Root Provider Tree
│   └── index.tsx           # Splash / Auth Check
├── src/
│   ├── components/         # Reusable UI components
│   │   ├── common/         # Buttons, Inputs, Avatars
│   │   ├── feed/           # PostCard, CommentModal
│   │   ├── map/            # MapOverlay, RiderMarker
│   │   └── chat/           # EncryptedMessageBubble
│   ├── contexts/           # React Context Providers
│   │   ├── AuthContext.tsx
│   │   ├── RideContext.tsx
│   │   └── E2EContext.tsx  # Handles Cryptography / Key Pairs
│   ├── services/           # API wrappers
│   │   ├── AuthService.ts
│   │   ├── FeedService.ts
│   │   ├── GarageService.ts
│   │   └── ChatService.ts
│   ├── hooks/              # Custom React Hooks
│   │   ├── useLocation.ts
│   │   └── useWebSocket.ts
│   ├── utils/              # Helper functions
│   │   ├── crypto.ts       # E2E Encryption/Decryption logic
│   │   └── formatters.ts
│   └── theme/              # Colors, Typography, Metrics
├── assets/                 # Images, Fonts, Icons
├── app.json                # Expo config
├── package.json
└── tsconfig.json
```

Backend Structure (/backend - Node.js)

```
/backend
├── src/
│   ├── config/              # Environment, DB, Cloudflare variables
│   │   ├── database.js
│   │   └── r2-storage.js
│   ├── controllers/         # Request handlers
│   │   ├── authController.js
│   │   ├── feedController.js
│   │   ├── rideController.js
│   │   └── chatController.js   # Handles storing/fetching encrypted payloads
│   ├── services/            # Core business logic
│   │   ├── authService.js
│   │   ├── geofenceService.js
│   │   └── telemetryService.js
│   ├── models/              # Sequelize DB Models
│   │   ├── index.js
│   │   ├── RiderAccount.js
│   │   ├── UserBike.js
│   │   ├── UserEncryptedChat.js
│   │   ├── FeedPost.js
│   │   └── Ride.js
│   ├── daos/                # Data Access Objects (DB Abstraction)
│   │   ├── riderAccountDao.js
│   │   └── rideDao.js
│   ├── routes/              # Express/Hapi Route Definitions
│   │   ├── index.js
│   │   ├── authRoutes.js
│   │   ├── feedRoutes.js
│   │   └── rideRoutes.js
│   ├── middlewares/         # Custom middlewares
│   │   ├── requireAuth.js
│   │   └── validateBody.js
│   ├── websockets/          # Live ride & live chat socket server
│   │   ├── wss.js           # Server initialization
│   │   ├── locationHandler.js  # Broadcasts GPS coordinates
│   │   └── chatHandler.js      # Relays E2E encrypted payloads live
│   └── utils/               # Helpers (Logger, Error formatters)
├── server.js                # Application Entry Point
├── .env.example
├── package.json
└── README.md
```

7. 22-Day Implementation Plan

Team Roles:

- **Dev 1 (Frontend):** React Native UI, State Management, Map integration, E2EE Client Crypto.
- **Dev 2 (Backend):** Node.js architecture, WebSockets, DB schemas, Encrypted DB storage.
- **Dev 3 (API/Testing):** API documentation (Postman), Edge-case testing, Cloudflare/VPS DevOps, DB Administration.

Phase 1: Foundation & Identity (Days 1 - 4)

- **Day 1:** * *Backend:* VPS setup, Install Postgres, Setup basic Node.js server.
  - *Frontend:* Expo router setup, Theme configuration, Asset loading.
  - *API:* Define Postman workspace, document Auth schemas.
- **Day 2:** * *Backend:* Firebase Auth validation endpoint, Sequelize User/Account models.
  - *Frontend:* Login/Signup UI, Splash screen, Key-pair generation for E2EE.
- **Day 3:** * *Backend:* Cloudflare R2 SDK integration for image uploads. Profile & Bike DAOs.
  - *Frontend:* Profile Setup UI, Add Bike UI.
- **Day 4:** * *Backend:* GET /me, POST /rider-account, POST /garage/bike.
  - *Frontend:* Wire up AuthContext. Complete E2E login to Dashboard flow.
  - *Testing:* Postman tests for Auth and Garage endpoints.

Phase 2: Social & Content (Days 5 - 9)

- **Day 5:**
  - *Backend:* Feed, Clips, and Comments DB schema.
  - *Frontend:* Home Feed UI (layout only).
- **Day 6:**
  - *Backend:* Feed CRUD APIs (POST /feed, POST /feed/images).
  - *Frontend:* Create Content Screen (Image picker, captions).
- **Day 7:**
  - *Backend:* Interactions APIs (Likes, Comments).
  - *Frontend:* Connect Home Feed to API. Implement pull-to-refresh.
- **Day 8:**
  - *Backend:* Social/Tracker APIs (Follow/Unfollow).
  - *Frontend:* Explore Screen UI, Search Users.
- **Day 9:**
  - *Frontend:* User Profile integration (Feed items, Stats layout).
  - *Testing:* API dev runs load tests on Cloudflare R2 uploads and Feed pagination.

Phase 3: Communities, Chat & Planning (Days 10 - 13)

- **Day 10:**
  - *Backend:* Communities & user_encrypted_chats DB schema.
  - *Frontend:* Communities List UI, Create Room UI.
- **Day 11:**
  - *Backend:* POST /community, Join/Leave logic.
  - *Frontend:* Room Details UI, E2E chat key exchange implementation.
- **Day 12:**
  - *Backend:* Route Planning endpoints via Google Maps APIs. Chat payloads API.
  - *Frontend:* Chat UI functioning (encrypting before send, decrypting on receive).
- **Day 13:**
  - *Backend:* Ride Model creation (Creating rides, adding participants).
  - *Testing:* Complete E2EE Chat integration tests, verify DB records are unreadable in plain text.

Phase 4: Live Ride Engine & Safety (Days 14 - 20)

- **Day 14:**
  - *Backend:* Setup WebSocket Server for Live Rides.
  - *Frontend:* Live Map Screen UI, Google Maps React Native implementation.
- **Day 15:**
  - *Backend:* WebSocket message handlers (broadcast rider_location).
  - *Frontend:* RideContext implementation, transmitting GPS updates to WS.
- **Day 16:**
  - *Backend:* Geofence calculation logic (background worker/service).
  - *Frontend:* Render other riders on Map from WS data.
- **Day 17:**
  - *Backend:* Alert Service (Out of Zone, Notify Captain).
  - *Frontend:* In-app UI Alerts (Red/Green status indicators).
- **Day 18:**
  - *Backend:* Ride End logic, Statistics generation (distance, speed).
  - *Frontend:* Ride summary screen.
- **Day 19:**
  - *Backend:* Notifications DB and Push Notification logic (Expo Push).
  - *Frontend:* Notifications Screen.
- **Day 20:**
  - *All:* Buffer day. Connect missing UI flows, refactor complex components, ensure all errors are caught and displayed nicely on the frontend.

Phase 5: QA & Delivery (Days 21 - 22)

- **Day 21:**
  - *API/Testing:* End-to-end Postman collection execution.
  - *Frontend:* UI/UX polishing across iOS and Android (Expo testing).
  - *Backend:* Database indexing optimization. Monitor WebSocket memory usage.
- **Day 22:**
  - *Team:* Field testing. (Mocking GPS locations with 3 devices to simulate a group ride).
  - Bug smashing.
  - Final deployment script preparation for the VPS.

8. Development Guidelines & Best Practices

1. **Security/E2EE:** Never log plain-text chat messages in backend consoles or error trackers. All decryption strictly happens locally on the client's device using securely stored keys.
2. **Error Handling:** The backend must always return standardized JSON errors { success: false, message: "...", code: "ERR_CODE" }. The frontend must use a global interceptor to catch these and show Toast messages, avoiding silent failures.
3. **Environment Variables:** Maintain .env.dev and .env.prod. Include keys for Firebase, Postgres, Cloudflare R2, and Google Maps.
4. **State Management:** Avoid putting everything in Redux/Context. Use AuthContext for user identity and RideContext for active map state. Local UI state should remain local to prevent unnecessary re-renders (crucial for map performance).
5. **Database:** Use Sequelize Migrations for every DB schema change. Do not sync models automatically in production.